EXPLAINABLE ACCESS CONTROL AND RISK-BASED AUTHORIZATION SYSTEM

A Course Project Report

Submitted in the partial fulfillment of the completion of the course

SOFTWARE ENGINEERING AND SYSTEM DESIGN

(24CS301PC311)

In III Year, I Semester B. Tech-CSE

Submitted by

M.Sai Abhigna

2403A51014

Under the supervision of the faculty

Dr.K.Deepa

<Assistant or Associate Professor>,

School Of Computer Science & Artificial Intelligence

2026-2027

Department of Computer Science & Artificial Intelligence

SR University

Warangal, Telangana, India

ABSTRACT

The Explainable Access Control and Risk-Based Authorization System is a web-based application designed to manage and control access to different resources based on user roles and risk scores.

The system uses Role-Based Access Control (RBAC) to assign permissions to different users such as Admin, Manager, and Employee. It also implements Risk-Based Authorization to evaluate access requests based on a risk score ranging from 0 to 100.

If the risk score is 70 or above, the system automatically denies access. If the risk score is below 70, access is granted or denied depending on the user's role and permissions.

The application is developed using Python Flask, HTML, CSS, and SQLite database. It provides features such as login authentication, access checking, access history, analytics dashboard, CSV report downloading, user management, and policy management.

The main objective of this project is to provide secure, transparent, and explainable access decisions.

INTRODUCTION

Access control is an important security mechanism used to protect sensitive information and resources in an organization.

Traditional access control systems mainly depend on fixed permissions. However, they may not consider the risk associated with an access request.

The Explainable Access Control and Risk-Based Authorization System combines role-based permissions with risk evaluation to make access decisions.

The system checks the user's role, requested resource, and risk score before granting or denying access. It also provides an explanation for every access decision.

This application helps demonstrate how access control mechanisms can improve security and transparency in an organization.

Objectives:

The main objectives of the Explainable Access Control and Risk-Based Authorization System are:

1. To develop a secure and user-friendly web-based access control application.

2. To implement Role-Based Access Control (RBAC) for managing permissions of different users.
3. To implement Risk-Based Authorization by evaluating access requests using risk scores.
4. To automatically deny access to high-risk requests to improve system security.
5. To provide explainable access decisions by displaying clear reasons for granting or denying access.
6. To maintain access request history for monitoring and auditing purposes.
7. To develop an analytics dashboard to visualize access requests, granted requests, denied requests, and risk levels.
8. To provide CSV report downloading for access history.
9. To display user roles and access policies through User Management and Policy Management modules.
10. To improve security, transparency, and accountability in access control systems.

Scope:

The scope of the Explainable Access Control and Risk-Based Authorization System is to provide a web-based application for managing and controlling access to organizational resources.

The system focuses on implementing role-based permissions and risk-based authorization to improve access security.

The scope of the project includes:

1. **User Authentication: Providing a login mechanism to access the application.**
2. **Role-Based Access Control: Managing resource permissions for Admin, Manager, and Employee roles.**
3. **Risk-Based Authorization: Evaluating access requests using risk scores ranging from 0 to 100.**
4. **Access Decision Explanation: Displaying reasons for granting or denying access.**
5. **Access History: Recording and displaying access requests and their decisions.**
6. **Analytics Dashboard: Displaying access statistics and risk-level analysis.**
7. **Report Generation: Providing access history in CSV format for downloading.**
8. **User Management: Displaying predefined users and their roles.**
9. **Policy Management: Displaying access permissions and risk policies.**

The project is developed as a prototype using Python Flask, HTML, CSS, and SQLite. It demonstrates basic access control and authorization concepts and can be extended with advanced security features in the future.

PROBLEM STATEMENT

Many traditional access control systems depend only on predefined user permissions. They may not consider the risk level associated with an access request.

This can create security problems when suspicious or high-risk access requests occur.

Therefore, a system is required that evaluates both user permissions and risk scores before granting access.

The proposed system addresses this problem by combining Role-Based Access Control and Risk-Based Authorization with explainable access decisions.

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Functional Requirements

Functional requirements describe the operations and features provided by the system.

The functional requirements of the Explainable Access Control and Risk-Based Authorization System are:

1. **Login Authentication:** The system shall allow the administrator to log in using valid credentials.
2. **Role-Based Access Control:** The system shall provide different access permissions for Admin, Manager, and Employee roles.
3. **Risk-Based Authorization:** The system shall evaluate access requests based on risk scores ranging from 0 to 100.
4. **Access Decision:** The system shall grant or deny access based on role permissions and risk scores.
5. **Explainable Decisions:** The system shall display explanations for access granted or access denied decisions.
6. **Access History:** The system shall store and display access request details, including username, role, resource, risk score, and decision.
7. **Analytics Dashboard:** The system shall display total requests, granted requests, denied requests, and risk-level statistics.

8. **CSV Download:** The system shall allow users to download access history in CSV format.
9. **User Management:** The system shall display predefined users, their roles, and account status.
10. **Policy Management:** The system shall display role-based access policies and risk-level policies.
11. **Logout:** The system shall allow users to log out and terminate their session.

Non-Functional Requirements

Non-functional requirements describe the quality, performance, and usability characteristics of the system.

1. **Security:** The system shall restrict access to application pages through login authentication.
2. **Usability:** The application shall provide a simple and user-friendly interface.
3. **Performance:** The system shall process access requests and display results with minimal delay under normal usage.
4. **Reliability:** The system shall maintain access history in the SQLite database.
5. **Maintainability:** The application shall use modular code that can be modified and extended easily.
6. **Availability:** The application shall be accessible whenever the Flask server is running.
7. **Explainability:** The system shall provide understandable reasons for access decisions.
8. **Data Integrity:** The system shall store access request information accurately in the database.

Hardware and Software Requirements

Hardware Requirements

Component	Minimum Requirement
Processor	Intel Core i3 or equivalent
RAM	4 GB
Storage	500 MB free space

Component	Minimum Requirement
System	Laptop or Desktop Computer
Display	Standard Monitor
Internet	Required for installation and setup

Software Requirements

Software	Purpose
Operating System	Windows 10 / Windows 11
Programming Language	Python 3.x
Framework	Flask
Frontend	HTML5, CSS3
Database	SQLite
IDE	Visual Studio Code
Web Browser	Google Chrome / Microsoft Edge
Database Connectivity	Python SQLite3 Module

Software	Purpose

SYSTEM ANALYSIS

Existing System

The existing access control systems mainly depend on predefined user roles and permissions to control access to resources. Traditional systems generally follow fixed access rules and may not consider the risk level associated with an access request.

In such systems, users may receive access based only on their assigned permissions. This approach may not effectively identify high-risk access requests.

Limitations of Existing System

- Limited consideration of risk factors.
- Less transparency in access decisions.
- Difficulty in identifying high-risk requests.
- Limited access monitoring and analysis.
- Lack of detailed explanations for access denial.
- Limited visualization of access statistics.

Proposed System

The proposed Explainable Access Control and Risk-Based Authorization System combines Role-Based Access Control (RBAC) with Risk-Based Authorization.

The system evaluates the user's role, requested resource, and risk score before making an access decision.

It supports three roles: Admin, Manager, and Employee. Access is granted only when role permissions are satisfied and the risk score is below 70.

The system also provides explanations for access decisions and maintains access history using an SQLite database.

Advantages of Proposed System

- Role-based resource access control.
- Risk-based access evaluation.
- Automatic denial of high-risk requests.
- Explainable access decisions.
- Access history and monitoring.

Software	Purpose

- Analytics dashboard.
- CSV report downloading.

SYSTEM DESIGN

System Architecture

The system follows a web-based architecture consisting of the frontend, backend, authorization logic, and database.

The frontend is developed using HTML and CSS. The backend uses Python Flask to process user requests and implement access control logic. SQLite stores access history and access decisions.

System Architecture Diagram

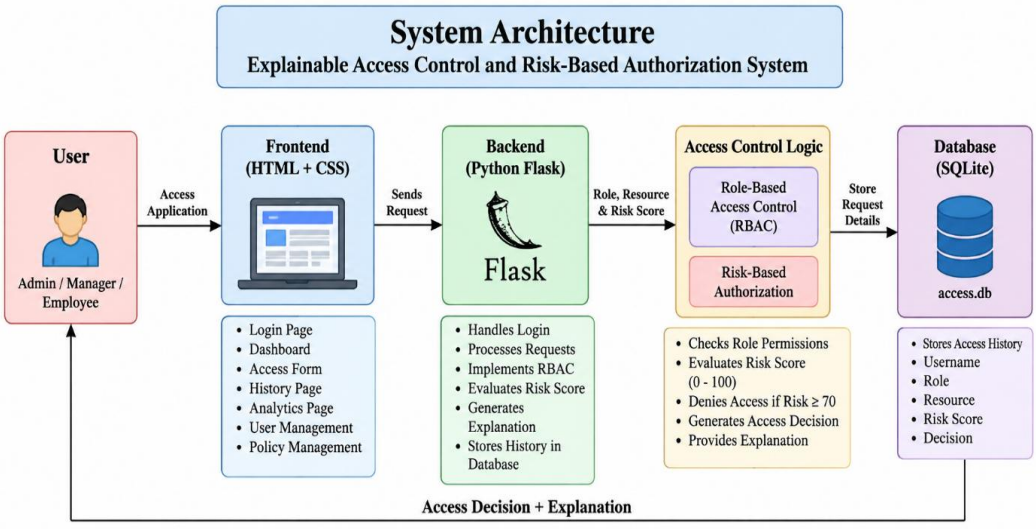


Figure 1: System Architecture of Explainable Access Control and Risk-Based Authorization System.

Use Case Diagram

The Use Case Diagram represents the interaction between the user and the system.

Actors

- Administrator
- User

Use Cases

- Login

Software **Purpose**

- Access Dashboard
- Check Access
- View Access Decision
- View Explanation
- View Access History
- View Analytics
- Download CSV Report
- View User Management
- View Policy Management
- Logout

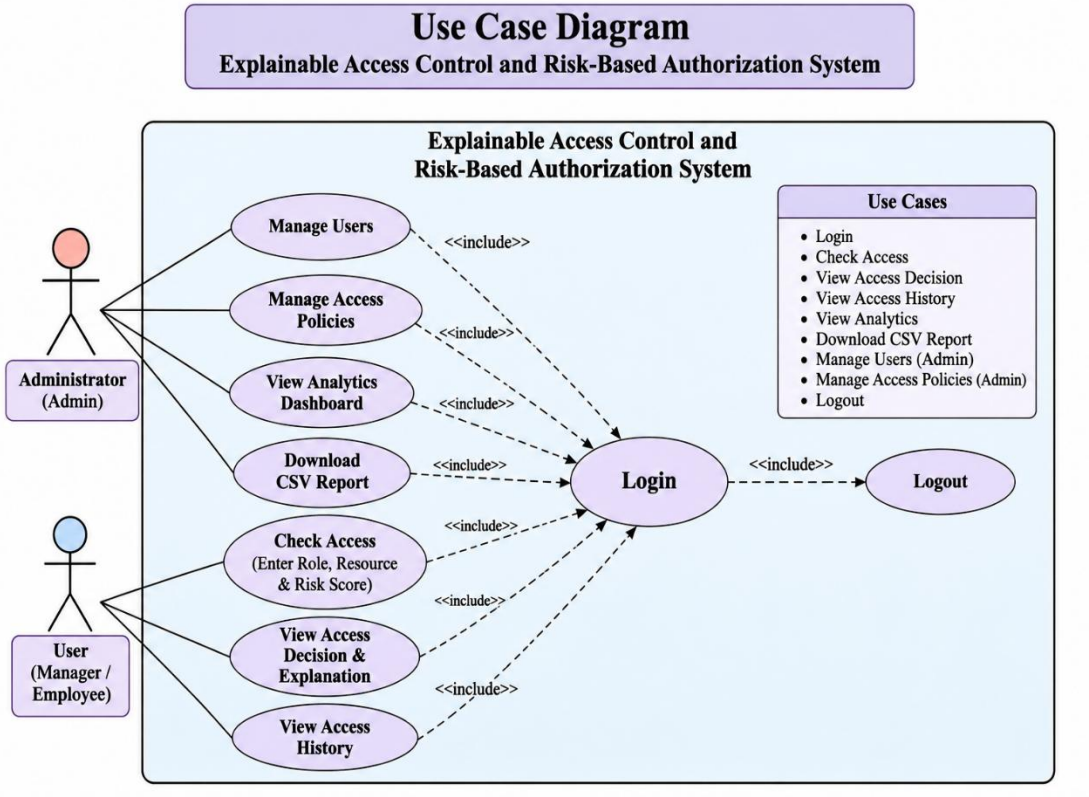


Figure 2: Use Case Diagram.

Class Diagram

The Class Diagram represents the structure of the system using classes, attributes, and methods.

Classes

Software **Purpose**

1. User

- username
- password
- role
- login()
- logout()

2. AccessRequest

- role
- resource
- risk
- decision
- checkAccess()

3. Authorization

- role
- risk
- evaluateRole()
- evaluateRisk()
- authorize()

4. AccessHistory

- username
- role
- resource
- risk
- decision
- saveHistory()
- viewHistory()

5. Analytics

Software **Purpose**

- totalRequests
- grantedRequests
- deniedRequests
- generateAnalytics()

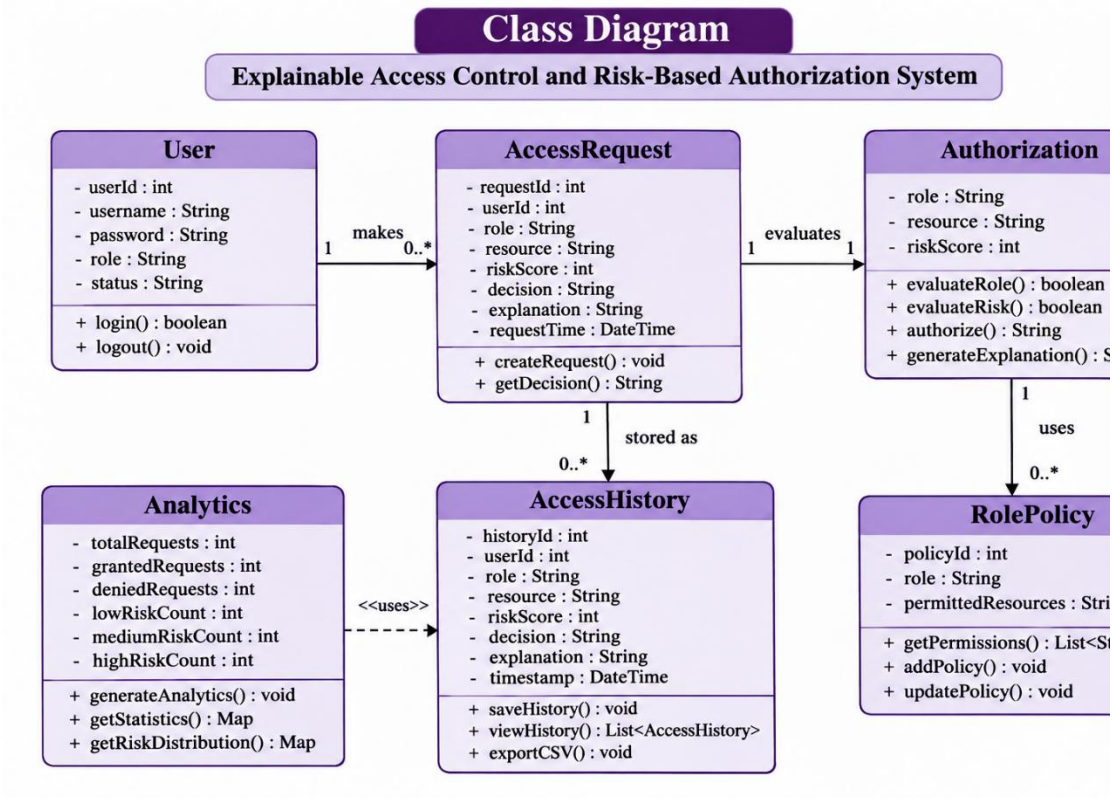


Figure 3: Class Diagram.

Sequence Diagram

The Sequence Diagram represents the sequence of interactions between the user, frontend, Flask application, authorization logic, and database.

Sequence of Operations

1. User enters login credentials.
2. Flask validates the credentials.
3. Dashboard is displayed.
4. User submits an access request.
5. Flask receives the request.
6. Authorization logic checks role permissions.

Software **Purpose**

7. Risk score is evaluated.
8. Access decision is generated.
9. Decision and explanation are displayed.
10. Access request is stored in SQLite.
11. User can view access history.

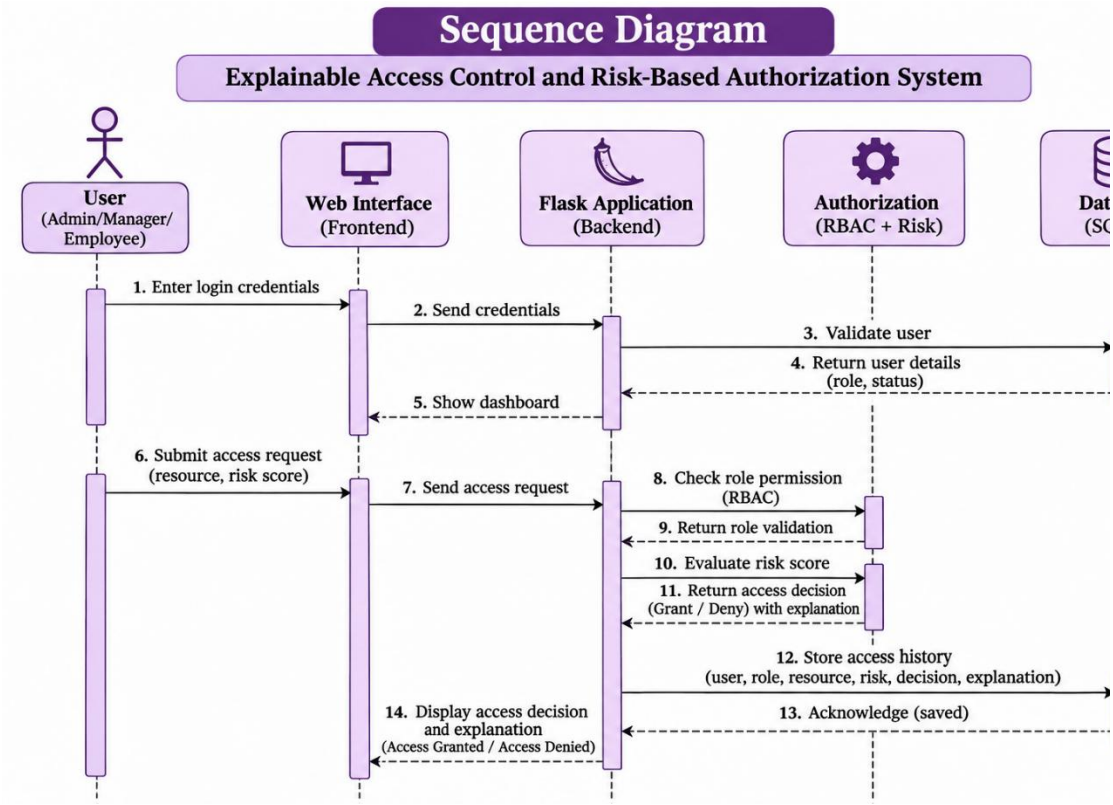


Figure 4: Sequence Diagram for Access Authorization.

Activity Diagram

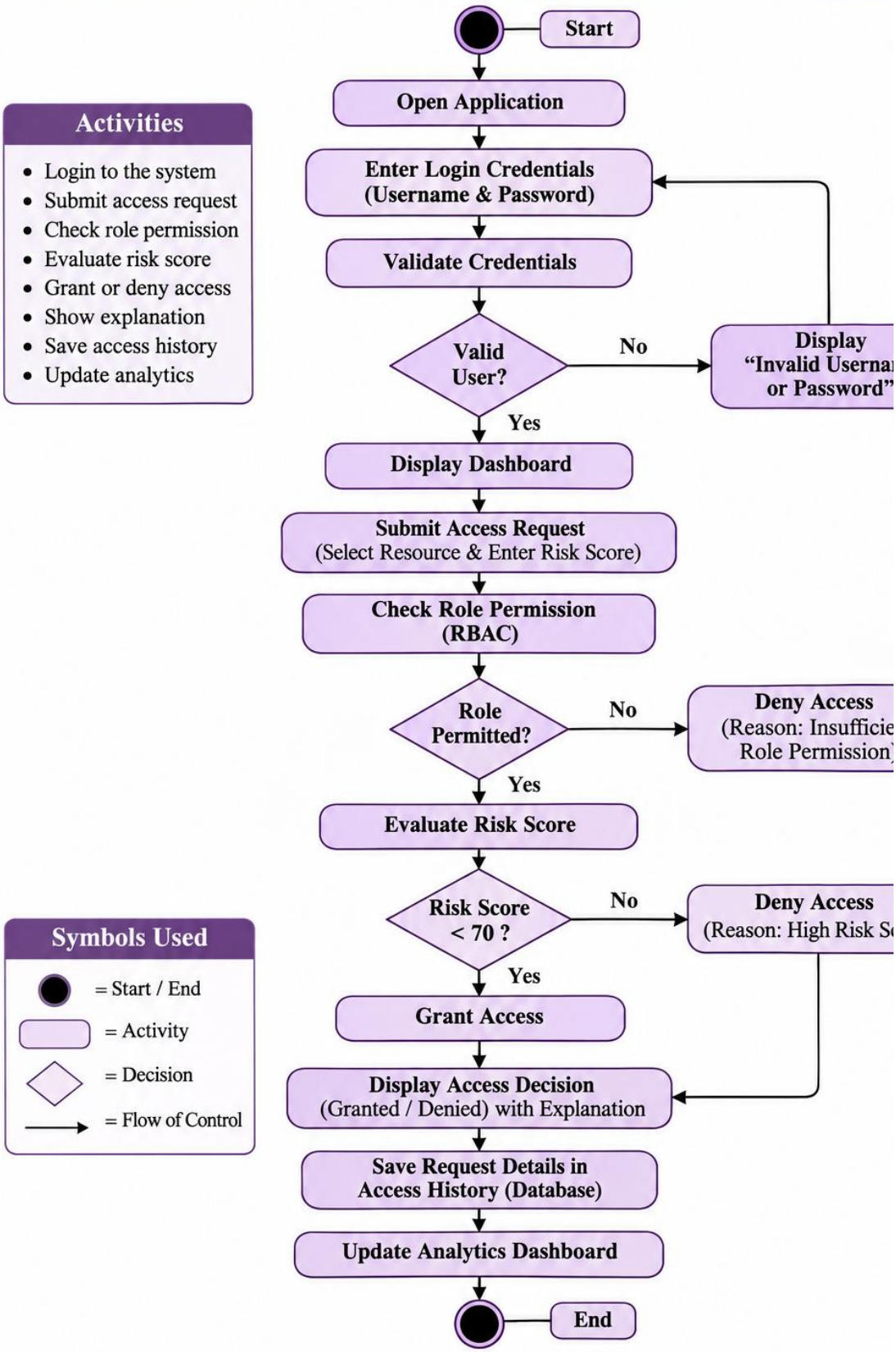
The Activity Diagram represents the workflow of the access authorization process.

Activity Flow

Start → Login → Validate Credentials → Dashboard → Enter Role, Resource, and Risk Score → Validate Risk Score → Check Role Permissions → Evaluate Risk → Access Granted / Access Denied → Display Explanation → Save Access History → End.

Activity Diagram

Explainable Access Control and Risk-Based Authorization System



Software	Purpose

Figure 5: Activity Diagram for Access Authorization.

METHODOLOGY

The methodology describes the development process followed to implement the Explainable Access Control and Risk-Based Authorization System.

Requirement Analysis

The functional and non-functional requirements of the application were identified. The required access control mechanisms and system features were determined.

System Design

The system architecture, use case diagram, class diagram, sequence diagram, and activity diagram were designed to represent the system structure and workflow.

Frontend Development

HTML and CSS were used to develop the login page, dashboard, access history, analytics dashboard, user management, and policy management interfaces.

Backend Development

Python Flask was used to implement login authentication, role-based access control, risk-based authorization, and explainable access decisions.

Database Implementation

SQLite was used to store access history, including username, role, resource, risk score, and access decision.

Testing

The application was tested using different roles, resources, and risk scores to verify the access control logic and expected results.

Deployment

The application was executed locally using the Flask development server and accessed through a web browser.

IMPLEMENTATION

The Explainable Access Control and Risk-Based Authorization System is implemented using Python Flask, HTML, CSS, and SQLite. The application is developed in Visual Studio Code and executed using the Flask development server.

Software	Purpose

Login Authentication Implementation

The login module provides authentication for accessing the application. The administrator enters a username and password. The Flask backend validates the credentials and creates a session after successful login. Invalid credentials display an error message.

Role-Based Access Control Implementation

The RBAC module checks the user's role before granting access to a requested resource. The system supports Admin, Manager, and Employee roles.

- Admin can access all resources.
- Manager can access Dashboard and Reports.
- Employee can access only Dashboard.

The system grants access only when the selected role has permission for the requested resource.

Risk-Based Authorization Implementation

The risk-based authorization module evaluates access requests using a risk score ranging from 0 to 100.

The system follows these risk levels:

- Low Risk: 0–39
- Medium Risk: 40–69
- High Risk: 70–100

If the risk score is 70 or above, access is automatically denied. Otherwise, access depends on the user's role permissions.

Explainable Access Decision Implementation

The explainable authorization module provides reasons for access decisions.

The system displays messages such as:

- Access allowed because role permissions and risk level are acceptable.
- Access denied because the role does not have permission.
- Access blocked because high risk was detected.

This improves transparency and helps users understand authorization decisions.

Access History Implementation

Software **Purpose**

The access history module uses SQLite to store access request details.

The stored information includes username, role, resource, risk score, and access decision.

Users can view previous access requests through the Access History page.

Analytics Dashboard Implementation

The analytics module retrieves access history from the SQLite database and displays access statistics.

It provides information about:

- Total access requests.
- Access granted requests.
- Access denied requests.
- Low-risk requests.
- Medium-risk requests.
- High-risk requests.

The dashboard helps monitor system activity and analyze access patterns.

CSV Download Implementation

The CSV download module allows users to export access history into a CSV file.

The Flask backend retrieves access records from SQLite and generates a downloadable file named access_history.csv.

This feature helps maintain access reports for monitoring and auditing purposes.

User Management Implementation

The User Management module displays predefined users and their corresponding roles and account statuses.

It provides a simple interface for demonstrating user information within the application.

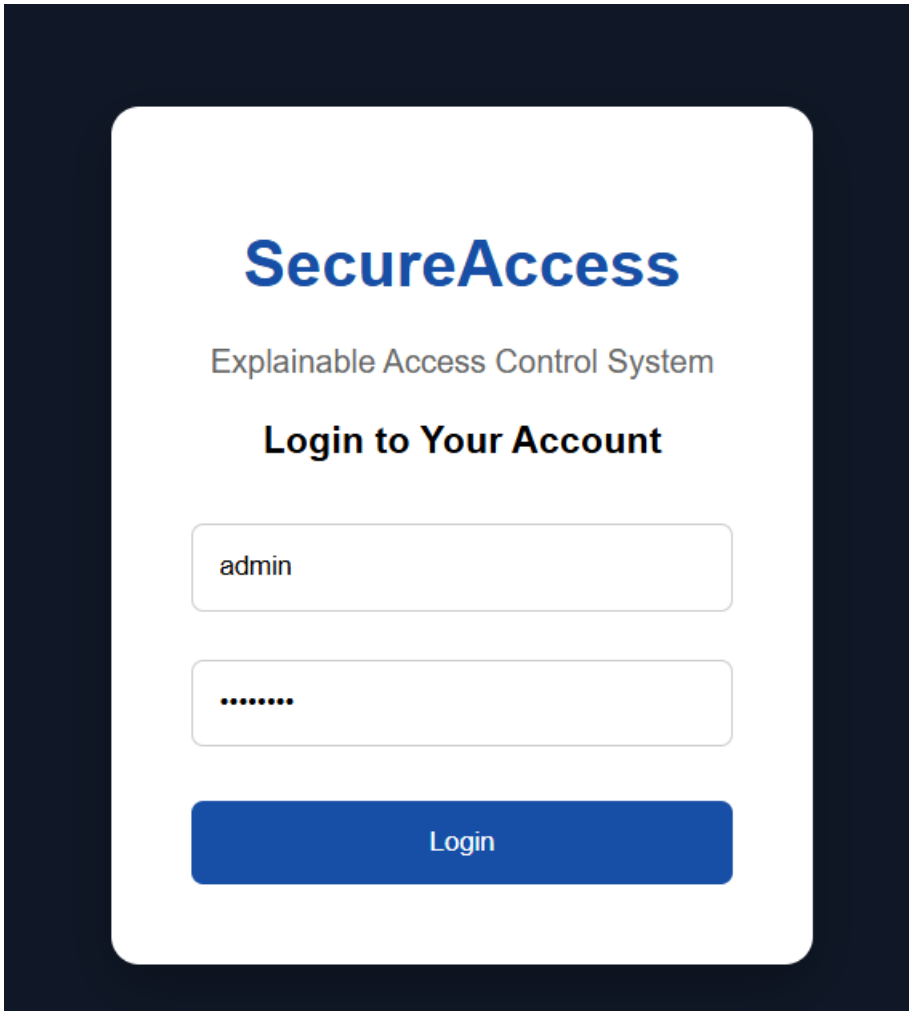
Policy Management Implementation

The Policy Management module displays predefined role permissions and risk-based access policies.

Software **Purpose**

It shows the resources accessible to Admin, Manager, and Employee roles and the risk thresholds used for authorization.

OUPUT SCREENSHOTS:



SecureAccess Dashboard

Explainable Access Control System

Role

Admin

Resource

Dashboard

Risk Score (0-100)

Check Access

Access History | Analytics | User Management

Policy Management | Logout

SecureAccess Dashboard

Explainable Access Control System

Role

Admin

Resource

Dashboard

Risk Score (0-100)

2

Check Access

Access History | Analytics | User Management

Policy Management | Logout

SecureAccess Dashboard

Explainable Access Control System

Role

Admin

Resource

Dashboard

Risk Score (0-100)

Check Access

ACCESS GRANTED

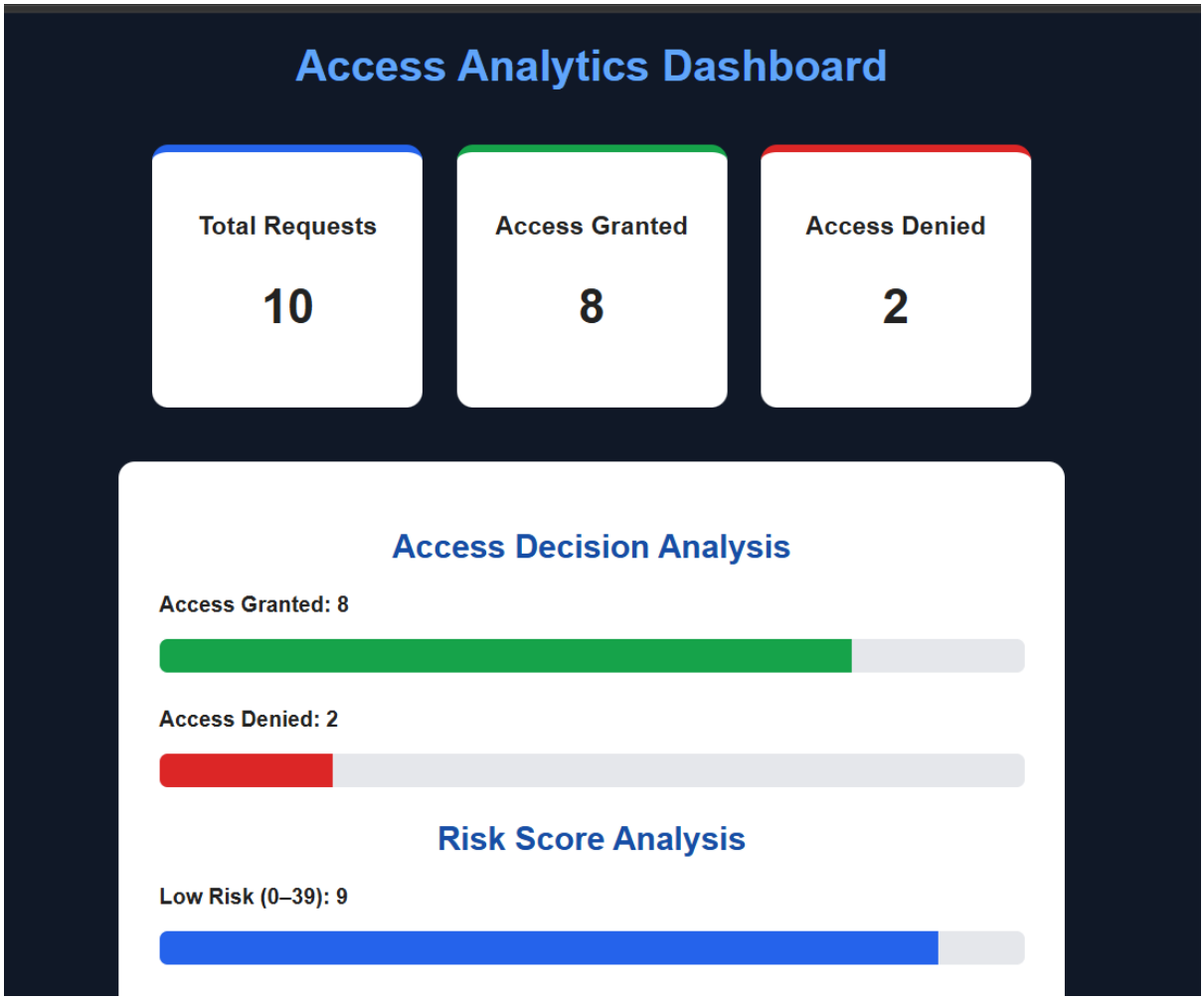
Access allowed. Role permissions and risk level are acceptable.

Access History | Analytics | User Management | Policy Management | Logout

Access History

Username	Role	Resource	Risk Score	Decision
admin	Admin	Dashboard	2	ACCESS GRANTED
admin	Admin	Dashboard	2	ACCESS GRANTED
admin	Admin	Dashboard	32	ACCESS GRANTED
admin	Admin	Dashboard	32	ACCESS GRANTED
admin	Admin	Dashboard	95	ACCESS DENIED
admin	Employee	User Data	20	ACCESS DENIED
admin	Admin	Dashboard	20	ACCESS GRANTED
admin	Admin	Dashboard	5	ACCESS GRANTED
admin	Admin	Dashboard	35	ACCESS GRANTED
admin	Admin	Dashboard	20	ACCESS GRANTED

Download Access History (CSV) Back to Dashboard

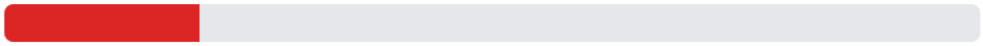

Access Analytics Dashboard
Total Requests
10
Access Granted
8
Access Denied
2
Access Decision Analysis
Access Granted: 8
Access Denied: 2
Risk Score Analysis
Low Risk (0–39): 9

Access Decision Analysis

Access Granted: 8



Access Denied: 2



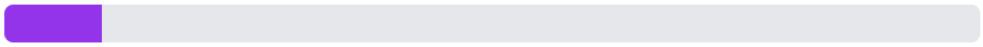
Risk Score Analysis

Low Risk (0–39): 9



Medium Risk (40–69): 0

High Risk (70–100): 1

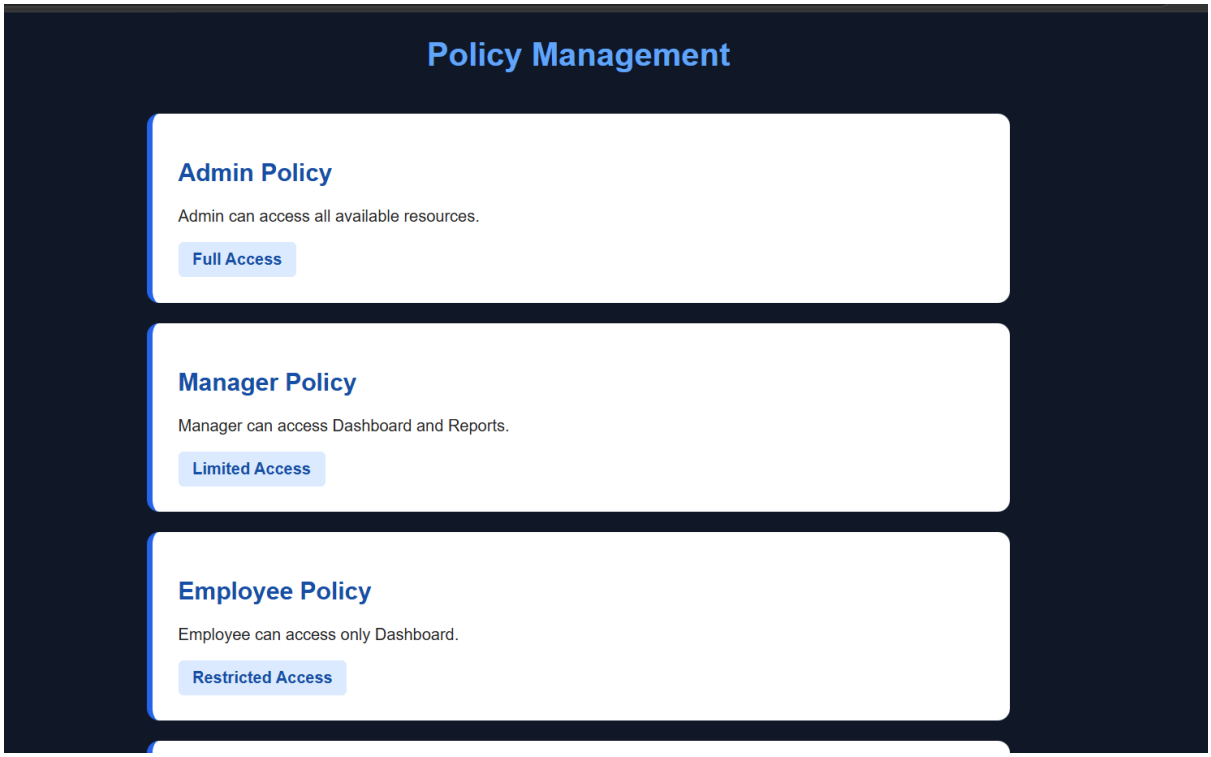


Back to Dashboard

User Management

Username	Role	Status
admin	Administrator	Active
manager	Manager	Active
employee	Employee	Active

Back to Dashboard



CONCLUSION

The Explainable Access Control and Risk-Based Authorization System was successfully developed as a web-based application using Python Flask, HTML, CSS, and SQLite.

The project combines Role-Based Access Control (RBAC) and Risk-Based Authorization to control access to organizational resources. It evaluates user roles, requested resources, and risk scores before making access decisions.

The system provides explainable access decisions, allowing users to understand why access is granted or denied. It also maintains access history, displays access statistics through an analytics dashboard, and supports CSV report downloading.

The project successfully demonstrates the importance of access control, risk evaluation, and transparency in application security.

Overall, this project provides a practical understanding of authorization mechanisms and serves as a foundation for developing more advanced security systems in the future.

REFERENCES

1. Python Official Documentation – https://docs.python.org/3/
2. Flask Official Documentation – https://flask.palletsprojects.com/

3. SQLite Official Documentation – https://www.sqlite.org/docs.html
4. W3Schools HTML Tutorial – https://www.w3schools.com/html/
5. W3Schools CSS Tutorial – https://www.w3schools.com/css/
6. OWASP – Access Control Security Guidelines – https://owasp.org/
7. Visual Studio Code Documentation – https://code.visualstudio.com/docs